


```

; CODE XMM from section '.text': 0x00000000
movss xmm1, 0x00000000
cmp byte [rdi + rax], 0 ; arg1
movss xmm1, 0x00000000
movss xmm1, 0x00000000
movss xmm1, 0x00000000
movss xmm1, 0x00000000
; CODE XMM from section '.text': 0x00000000
movss xmm1, 0x00000000
movss xmm1, 0x00000000

```

```

--(agonist@attack)-[~/ctf/cracking/rootme/ELF x64 - Basic KeygenMe]
_ $ echo -e "root-me.org" | ./cl36.pln
[1] x64 NACM Keygen-Me
[2] root-me
[3] Login :
[4] Yeah, good job bro, now write a keygen :)
--(agonist@attack)-[~/ctf/cracking/rootme/ELF x64 - Basic KeygenMe]
_ $

```

```

--(agonist@attack)-[~/ctf/cracking/rootme/ELF x64 - Basic KeygenMe]
_ $ echo -e "\x00\x02\x01\x05" | ./cl36.pln
--(agonist@attack)-[~/ctf/cracking/rootme/ELF x64 - Basic KeygenMe]
_ $ sha256sum .n.key1
c5d0d9f9c211425f1062492e0760d0427809eb08aac06100f64ad022c26 .n.key1

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
//cmp rcx,rcx ; ==> si rcx est égale à rcx on saute faire la fin de la routine
.text:0000000000400164 jz short loc_400182
.text:0000000000400166 mov dl, [rdi+rcx] ; ==> ici si on parle du principe que rcx = 0 alors [rdi+rcx] = input[0]
.text:0000000000400169 mov dh, [rsi+rcx] ; ==> rsi contient : 000200h donc [rsi+rcx] = 0
.text:000000000040016C mov al, dl ; ==> on met le byte de dl dans al
.text:000000000040016E sub al, cl ; ==> on soustrait le byte de dl avec le byte de la valeur contenue dans rcx (rcx est un compteur)
.text:0000000000400170 add al, 14h ; on additionne la valeur soustraite avec 20 en décimal
.text:0000000000400172 cmp al, dh ; ==> on compare le byte contenue dans dh (rdx??) avec celui de al (eax)
.text:0000000000400174 jnz short loc_40017B ; ==> si ce n'est pas égal on saute
.text:0000000000400176 inc rcx ; ==> sinon on incrémente le compteur rcx
.text:0000000000400179 jmp short loc_400161 ; ==> et on recommence*/
//on sait que notre input doit faire une taille de 32 caractères
//on sait qu'il ne faut pas de xor
//dl 8-7 Byte BAS Premier octet
//dh 8-15 Byte HAUT Deuxième octet
//cl[1] = input[1] - i + 20
int main()
{
    printf("KEYGEN by 4LIP\n");
    unsigned char input[] = "root-me.org";
    unsigned char key [10] = {0};
    for(int i = 0; i < 10; i++){
        key[i] = input[i] - i + 20;
    }
    printf("voici la cle\n");
    for(int z = 0; z < 10; z++){
        printf("%02x ", key[z]);
    }
    return 0;
}

```